

Project 02: Build a REST API

Motivation and Description

Use the agile process to research, design, develop, test, and deploy a production grade REST API. This project gives you experience with the full lifecycle of a backend service: contract design, database modeling, authentication, containerization, hosting, testing, and documentation.

Teams will design and implement a **REST API** using **Spring Boot**, backed by a **remote hosted database**, secured with **OAuth2**, containerized with **Docker**, and deployed to a public host. A small **Kotlin Android** app serves as the client that proves the API works end to end.

Two things are different from Project 01, and they are the point of this project:

1. **You design the API before you build it.** You will hand write your OpenAPI contract in Sprint 1, review it, and only then write controllers. The contract is a deliverable, not a byproduct.
2. **Testing is not a final week activity.** Coverage is enforced by your build, and your CI runs on every pull request.

The Android app is the demo, not the deliverable. It is deliberately scoped small (four screens). Polish earns no points. The API is where your effort belongs.

Table of Contents

[Project 02: Build a REST API](#)

[Motivation and Description](#)

[Table of Contents](#)

[Choose Your API Domain](#)

[Technology](#)

[Agile Process](#)

[Design First: The Contract](#)

[Project Requirements](#)

[Testing](#)

[Documentation](#)

[Things You Will Need to Figure Out \(In Order\)](#)

[Sprints and MVP Milestones](#)

[Grading Rubric](#)

[Helpful Links](#)

[Frequently Asked Questions](#)

[Changelog](#)

Choose Your API Domain

Pick one of the following, or propose your own. A proposal needs instructor approval in Week 1 (one paragraph, a list of endpoints, and an ER sketch). Whatever you choose must support users, at least two other resource types, and meaningful relationships between them.

1. **Event Management.** Users create events, send invitations, track RSVPs.
2. **Fitness and Workout Tracker.** Users log sessions, track exercises and sets, view progress.
3. **Recipe and Meal Planner.** Users create recipes, add ingredients, build weekly plans.
4. **Job Application Tracker.** Users track applications, companies, interview stages, notes.
5. **Study Group Finder.** Students post study sessions by course, location, and time.
6. **Something else.** Bring a proposal in Week 1.

Technology

Back End (required)

Spring Boot with **Gradle**. Use Gradle for both repos so you only learn one build tool.

- Spring Boot quickstart: <https://spring.io/quickstart>
- Building a REST service: <https://spring.io/guides/gs/rest-service/>
- Spring Data JPA: <https://spring.io/guides/gs/accessing-data-jpa/>
- Gradle guides: <https://gradle.org/guides/>

Front End (required)

Kotlin Android, built with Jetpack Compose. The app runs on an emulator or a device and talks to your live API. It does not need to be published anywhere.

- Compose documentation: <https://developer.android.com/develop/ui/compose/documentation>
- Android Basics with Compose course: <https://developer.android.com/courses/android-basics-compose/course>
- Recommended app architecture: <https://developer.android.com/topic/architecture>
- Retrofit (HTTP client): <https://github.com/square/retrofit>
- OkHttp: <https://github.com/square/okhttp>

Database (required: remote and hosted)

Every team member must be able to connect to the same live database.

- Supabase, PostgreSQL: <https://supabase.com/>
- MongoDB Atlas: <https://www.mongodb.com/cloud/atlas>
- Railway, PostgreSQL or MySQL: <https://railway.app/>
- Render: <https://render.com/>

Do **not** use a local SQLite file as your production database.

Do **not** put the database inside your Docker container.

Agile Process

GitHub Workflow

- Use a GitHub Project board (Kanban) to track work:
<https://docs.github.com/en/issues/planning-and-tracking-with-projects/learning-about-projects/about-projects>
- Separate repos for the API and the Android app, or one monorepo with two top level directories. Your call.
- **All merges to main go through a pull request.** Set branch protection in Week 1, not Week 4.
- Every PR must be reviewed and approved by **2 teammates who are not the author**. If your team has three people, 1 approving reviewer is enough.
- Every PR must reference at least one issue.
- Issues are user stories: *"As a [role], I want to [action] so that [benefit]."*

User Stories

Write at least **5 issues per team member** (a team of 4 needs 20, a team of 3 needs 15).

Each issue must:

- Be in user story format
- Have a clear acceptance criterion, meaning anyone on the team can tell when it is done
- Be attached to a Sprint milestone

Milestones are how I evaluate your agile process, so keep them current.

Design First: The Contract

This is the part of the project most teams will want to skip. Do not skip it. Every hour here saves three in Sprint 3.

Write the OpenAPI spec by hand, in Sprint 1, before you write controllers

You will hand write an `openapi.yaml` describing every endpoint you plan to build: paths, verbs, request bodies, response bodies, and status codes. Commit it at the repo root.

Why this order matters:

- It forces the design conversation before the code makes the decision for you.
- Your Android developers can generate a client and start against a mock server immediately, instead of waiting on the backend.
- In Sprint 4, SpringDoc will publish docs from your annotated code. If those docs disagree with the contract you committed in Sprint 1, that is a finding worth discussing in your retro.

Tools:

- OpenAPI specification reference: <https://swagger.io/specification/>
- Swagger Editor, paste your YAML and see it render: <https://editor.swagger.io/>

Design review at the end of Sprint 1

Your team presents its contract and ER diagram to another team and to me. Twenty minutes. We will look at resource naming, verb semantics, status codes, the error shape, pagination, and what happens on delete. You practiced code review in Project 01. This is the same skill applied earlier, where it is cheaper.

Required design artifacts

All of these are committed to the repo, in a `/docs` directory.

Artifact	What it must show
<code>openapi.yaml</code>	Every planned endpoint, hand written, before controllers exist
ER diagram	Tables, primary and foreign keys, cardinality, and which fields are nullable
Auth sequence diagram	The OAuth2 PKCE flow, every hop, from app launch to an authorized API call
Error model	One consistent error body used by every endpoint
2 ADRs	One page each, for two real decisions (database choice, auth provider, monorepo or split)

Diagram tools, use whatever you like:

- Mermaid ER diagrams, renders natively in GitHub: <https://mermaid.js.org/syntax/entityRelationshipDiagram.html>
- Mermaid sequence diagrams: <https://mermaid.js.org/syntax/sequenceDiagram.html>
- dbdiagram.io: <https://dbdiagram.io/>
- What an ADR is: <https://adr.github.io/>
- ADR examples and templates: <https://github.com/joelparkerhenderson/architecture-decision-record>

API design decisions you must make and document

These are the judgment calls that separate an API from a pile of routes. Each one gets a sentence in your README or your spec.

- **Pagination.**
 - At least one collection endpoint must paginate.
 - An endpoint that returns every row is a design smell. Say whether you chose page or cursor based, and why.
- **Filtering or sorting.** At least one collection must support it.
- **Idempotency.** PUT is idempotent, POST is not. Say where that distinction matters in your API.
- **Versioning.** Prefix your routes (`/api/v1/...`) and write one sentence on why versioning exists.
- **Error shape.** Pick one and use it everywhere. RFC 9457 Problem Details is the standard and Spring supports it out of the box.
 - RFC 9457: <https://www.rfc-editor.org/rfc/rfc9457>
 - Spring support: <https://docs.spring.io/spring-framework/reference/web/webmvc/mvc-ann-rest-exceptions.html>

Project Requirements

- ☐ REST API built with Spring Boot
- ☐ `openapi.yaml` hand written and committed in Sprint 1
- ☐ ER diagram, auth sequence diagram, error model, and 2 ADRs in `/docs`
- ☐ All five REST verbs used with non-trivial routes
- ☐ Pagination on at least one collection endpoint
- ☐ Remote hosted database, not local, not in the container
- ☐ OAuth2 Authorization Code with PKCE, API acts as a Resource Server
- ☐ Admin role stored in the database and enforced server side
- ☐ Multi-stage `Dockerfile` plus `.dockerignore`, image builds and runs the application
- ☐ `docker-compose.yml` brings up the API and a local database for development
- ☐ CI builds the Docker image on every pull request
- ☐ Image published to the GitHub Container Registry on merge to main
- ☐ No secrets baked into the image
- ☐ API live at a public HTTPS URL
- ☐ Kotlin Android app with four screens, exercising the API
- ☐ JUnit 5 unit tests with 70 percent line coverage on the service layer, enforced by the build
- ☐ REST Assured integration tests using Testcontainers
- ☐ Android tests: ViewModel, MockWebServer, and one Compose UI test
- ☐ GitHub Actions runs tests on every pull request, as a required check
- ☐ Swagger UI reachable on the live deployment
- ☐ README in each repo
- ☐ KDoc or Javadoc on all public classes and methods
- ☐ Four sprint retrospectives

REST API Requirements

There must be at least one meaningful route for each verb. A route that exists only to tick a box does not count.

Verb	Typical Use
GET	Retrieve a resource or a collection
POST	Create a new resource
PUT	Replace a resource in full
PATCH	Partially update a resource
DELETE	Remove a resource

All responses use appropriate HTTP status codes. Keep this open while you work:

<https://developer.mozilla.org/en-US/docs/Web/HTTP/Status>

A note on CORS. Project 01 and the previous version of this project required CORS configuration. CORS is a *browser* security policy. Your Android client does not enforce it, so it is no longer a requirement and no longer worth points. You may still need a small CORS config if you want to call your API from Swagger UI or a browser tool, but that is your choice.

Authentication: OAuth2 with PKCE

Read this section carefully. It is the most common place teams go wrong, and the setup is different from what you may have seen in web tutorials.

A native mobile app cannot keep a client secret, and it has no browser session to hold a cookie. So the web pattern (Spring Boot as an OAuth2 *Client*, handling a redirect and setting a session cookie) does not work here. You will use the mobile pattern instead:

- The **Android app** runs the **Authorization Code flow with PKCE**. It opens a Chrome Custom Tab, the user signs in with Google, and the app receives an authorization code and exchanges it for a token.
- The app sends that token on every request as **Authorization: Bearer <token>**.
- The **Spring Boot API** is configured as an **OAuth2 Resource Server**. It validates the token on each request. It does **not** manage a login page or a session.

Concretely, your API depends on **spring-boot-starter-oauth2-resource-server**, **not** **spring-boot-starter-oauth2-client**.

- What PKCE is and why mobile needs it: <https://oauth.net/2/pkce/>
- The specification, RFC 7636: <https://datatracker.ietf.org/doc/html/rfc7636>
- Google's guidance for native apps:
<https://developers.google.com/identity/protocols/oauth2/native-app>
- Google OAuth2 setup: <https://developers.google.com/identity/protocols/oauth2>
- Spring Security Resource Server, JWT:
<https://docs.spring.io/spring-security/reference/servlet/oauth2/resource-server/jwt.html>
- Resource Server overview:
<https://docs.spring.io/spring-security/reference/servlet/oauth2/resource-server/index.html>
- Spring Boot security guide:
<https://docs.spring.io/spring-boot/reference/web/spring-security.html>
- AppAuth for Android, a well maintained PKCE client:
<https://github.com/openid/AppAuth-Android>
- Credential Manager with Sign in with Google:
<https://developer.android.com/identity/sign-in/credential-manager-siwg>

Requirements:

- Protected routes return **401** when there is no valid token
- Routes requiring a role return **403** when the user lacks it
- The admin role lives in your database, not in a hardcoded list
- Method level security reference:
<https://docs.spring.io/spring-security/reference/servlet/authorization/method-security.html>

You do not build a login form and you do not store passwords.

Admin Features

Admins can do everything a user can, plus:

- `GET /api/v1/users` list all users
- `GET /api/v1/users/{userId}` view one user
- `PATCH /api/v1/users/{userId}` update a user, for example grant or revoke admin
- `DELETE /api/v1/users/{userId}` delete a user and all their data

Android Front End (four screens)

Scope is deliberately small. Build these four and stop. Extra screens, animation, and theming earn nothing.

1. **Login.** Sign in with Google via PKCE. On success, go to the list screen.
2. **Resource list.** The primary resource for your domain, loaded from the live API, paginated.
3. **Resource detail.** View, edit, and create a single resource. One screen, not three.
4. **Admin.** Visible only to admins. List users, toggle the admin role, delete a user.

Account deletion can be a menu item or a button on any screen. It does not need its own screen.

The app must show something useful for **401, 403, 404, and 500**. A blank screen on error is not acceptable.

Two things that will cost you an afternoon if you do not know them now:

- From the Android emulator, your development machine is **10.0.2.2**, not `localhost` or `127.0.0.1`. <https://developer.android.com/studio/run/emulator-networking>
- Android blocks cleartext HTTP by default. Your deployed API must be **HTTPS**. For local development against plain HTTP you need a network security configuration. <https://developer.android.com/privacy-and-security/security-config>

Deliverable for the app: a debug APK attached to a GitHub release, **or** a screen recording that walks through all four screens against your live API.

Persistence Layer

- At least **three tables or collections**: users plus two resource types
- At least one **one to many** or **many to many** relationship
- All application data persists across sessions
- Do not store image binaries in the database, store a URL

- When a user deletes their account, their data is actually deleted
- Spring Boot SQL database reference:
<https://docs.spring.io/spring-boot/reference/data/sql.html>

Docker

Docker is not just how you ship this project. It is how you develop it, how you test it, and how CI checks it. You will use it in four places.

1. A multi-stage `Dockerfile` that builds the API. Stage one builds with Gradle, stage two copies only the resulting jar into a slim runtime image. Your final image must not ship the Gradle toolchain or your source tree. Commit a `.dockerignore` so build context stays small and nothing stray gets baked in.

2. A `docker-compose.yml` for local development. One `docker compose up` should start your API plus a local Postgres, so a teammate who just cloned the repo can run the whole stack without installing a database. This local Postgres is for development only. Your deployed API still points at the remote hosted database.

3. Testcontainers, which is Docker, in your integration tests. The Testcontainers requirement in the Testing section is a Docker requirement. Your integration tests start a real Postgres in a throwaway container, run against it, and tear it down. That is why those tests behave the same on your machine and in CI.

4. CI builds the image on every pull request. A `docker build` step in GitHub Actions means a broken Dockerfile fails in minutes, on the PR, instead of on deploy day in Sprint 3. On merge to main, publish the image to the GitHub Container Registry and have your host deploy that tag.

Secrets never go in the image. Database URLs, passwords, and OAuth client secrets are environment variables supplied at runtime by your host, or GitHub Actions secrets in CI. If a credential is in your `Dockerfile` or committed to the repo, that is an automatic deduction, and you should rotate it.

Also required:

- The containerized API is deployed to a public HTTPS URL
- The live URL is in the README
- The database runs outside the container
- A `HEALTHCHECK` in your Dockerfile, or a Spring Actuator health endpoint your host can poll

Links:

- Docker getting started: <https://docs.docker.com/get-started/>
- Dockerfile reference: <https://docs.docker.com/reference/dockerfile/>

- Multi-stage builds: <https://docs.docker.com/build/building/multi-stage/>
- Dockerfile best practices: <https://docs.docker.com/build/building/best-practices/>
- `.dockerignore`: <https://docs.docker.com/build/concepts/context/#dockerignore-files>
- Docker Compose getting started: <https://docs.docker.com/compose/gettingstarted/>
- Spring Boot with Docker: <https://spring.io/guides/gs/spring-boot-docker/>
- Publishing an image with GitHub Actions:
<https://docs.github.com/en/packages/managing-github-packages-using-github-actions-workflows/publishing-and-installing-a-package-with-github-actions>
- `docker/build-push-action`: <https://github.com/docker/build-push-action>
- GitHub Actions secrets: <https://docs.github.com/en/actions/concepts/security/secrets>
- Spring Actuator endpoints:
<https://docs.spring.io/spring-boot/reference/actuator/endpoints.html>

Testing

Testing is 20 points, the largest single category after the API itself. Write tests as you go. A team that leaves testing until Sprint 4 will not finish.

Unit tests, JUnit 5

- Test service and business logic independently from the database. Mock the repository layer.
- **Coverage floor: 70 percent line coverage on your service layer, enforced by the build.** Configure the JaCoCo Gradle plugin so that a build below the threshold fails. This is not a suggestion you can satisfy at the end; wire it up in Sprint 2.
- Coverage is necessary but not sufficient. Tests that assert a getter returns what you set are not meaningful and will not earn the qualitative points. Test the decisions your code makes: does the service reject invalid input, does it refuse to return another user's resource, does it handle the empty case.
- JUnit 5 user guide: <https://junit.org/junit5/docs/current/user-guide/>
- Mockito: <https://site.mockito.org/>
- Spring Boot testing reference: <https://docs.spring.io/spring-boot/reference/testing/index.html>
- Testing Spring Boot applications: <https://docs.spring.io/spring-boot/reference/testing/spring-boot-applications.html>
- Testing the web layer: <https://spring.io/guides/gs/testing-web/>
- JaCoCo Gradle plugin, including coverage verification: https://docs.gradle.org/current/userguide/jacoco_plugin.html

Integration tests, REST Assured and Testcontainers

Integration tests hit your real endpoints. Run them against a **Testcontainers** database, not your production database and not an in memory substitute that behaves differently from Postgres.

Testcontainers **is Docker**. It starts a real Postgres in a throwaway container for the duration of your test run and destroys it afterwards. This is the same skill as the rest of your Docker work, applied to testing, and it is the reason these tests give the same answer on your laptop and in CI. Docker must be available on the CI runner; the GitHub Actions Ubuntu runners have it.

Required paths, per endpoint group:

- One **happy path**
- **401** unauthenticated
- **403** authenticated but wrong role
- **404** resource does not exist
- **400** invalid request body
- REST Assured: <https://rest-assured.io/>
- REST Assured usage guide: <https://github.com/rest-assured/rest-assured/wiki/Usage>
- Testcontainers for Java: <https://java.testcontainers.org/>
- Testcontainers Postgres module:
<https://java.testcontainers.org/modules/databases/postgres/>
- Spring Boot Testcontainers support:
<https://docs.spring.io/spring-boot/reference/testing/testcontainers.html>
- Spring profiles, for a test profile:
<https://docs.spring.io/spring-boot/reference/features/profiles.html>

Android tests

The client carries tests too.

- **ViewModel unit tests.** State transitions and error handling, using the coroutines test library. <https://developer.android.com/kotlin/coroutines/test>
- **API client tests with MockWebServer.** Assert the request path and headers your client sends, and that the JSON maps correctly into your models. Include a 401 and a 500 response. <https://github.com/square/okhttp/tree/master/mockwebserver> These are contract tests: they check your client against the same spec the API implements.
- **One Compose UI test**, for the login screen or the list screen.
<https://developer.android.com/develop/ui/compose/testing> Cheat sheet:
<https://developer.android.com/develop/ui/compose/testing/testing-cheatsheet>
- Local unit tests on Android: <https://developer.android.com/training/testing/local-tests>

CI

- GitHub Actions runs your test suite on **every pull request**, not only on pushes to main. By the time it is on main it is too late.
 - Make the check **required** in branch protection so a red build blocks the merge.
 - GitHub Actions documentation: <https://docs.github.com/en/actions>
 - Build and test Java with Gradle:
<https://docs.github.com/en/actions/tutorials/build-and-test-code/java-with-gradle>
-

Documentation

Swagger and OpenAPI

Your hand written `openapi.yaml` is the design artifact. SpringDoc publishes live docs from your annotated controllers. Both exist, and by Sprint 4 they should agree.

- Reachable at `/swagger-ui.html` on the live deployment
- Controllers annotated with `@Operation` and `@ApiResponse`
- SpringDoc: <https://springdoc.org/>

README

Each repo needs a **README.md** with the project name and description, team members, tech stack, step by step local setup including Docker commands, the live API URL, and a link to the Swagger docs.

KDoc and Javadoc

All public classes and public methods are documented. Kotlin uses KDoc, Java uses Javadoc. Both are generated by Gradle.

Note: Postman collections are no longer required. Your OpenAPI spec covers the same ground and will not drift from the code. If you like Postman, import your spec into it:

<https://www.postman.com/>

Things You Will Need to Figure Out (In Order)

Roughly in this order. Do not leave Docker, the database, or auth until late.

1. **Decide what data your app needs.** Talk as a team. Draw the ER diagram before any code.
2. **Write the OpenAPI contract by hand.** Argue about it now, not in Sprint 3.
3. **Provision the remote database** and connect Spring Data to it.
4. **Get OAuth2 PKCE working end to end early.** A login button in the Android app that produces a token, and one protected endpoint that accepts it. Do this before you have a dozen other new things to debug alongside it.
5. **Build API routes incrementally.** Start with one GET returning real data.
6. **Containerize early.** A small app is far easier to dockerize than a finished one. Get the multi-stage build and `docker compose up` working in Sprint 2, while there is little to break.
7. **Deploy early.** A half working live API beats a perfect local one.
8. **Write tests as you build.** Every endpoint ships with its tests. Wire up JaCoCo and CI in Sprint 2 so the floor is enforced from then on.
9. **Fill in the Android screens.** The vertical slice already exists from step 4; the remaining screens are mostly more of the same.

Sprints and MVP Milestones

Four one week sprints. Each ends with a short written retrospective submitted to me: what went well, what did not, what you are changing.

Sprint	Expected state
Sprint 1	Domain approved. <code>openapi.yaml</code> , ER diagram, and error model committed. Design review done. Remote DB connected. Skeleton Spring Boot project with one GET endpoint. GitHub Project populated.
Sprint 2	Three or more endpoints returning real data. OAuth2 PKCE working end to end: Android login produces a token, one protected endpoint accepts it. Multi-stage Docker build and <code>docker compose up</code> both working locally. CI running on PRs, building the image and enforcing JaCoCo.
Sprint 3	All five verbs live on the public HTTPS host, deployed from a published image. Admin endpoints working. Unit tests passing at the 70 percent floor. REST Assured integration tests with Testcontainers covering happy and error paths. Auth sequence diagram committed.
Sprint 4	Android app complete, all four screens against the live API. Android tests passing. Swagger UI live. READMEs and KDoc complete. ADRs committed. All four retrospectives submitted.

Note what moved compared to previous versions of this project: **the auth slice is in Sprint 2 and integration tests are in Sprint 3.** Sprint 4 is for finishing the client and the documentation, not for discovering that your auth does not work.

Grading Rubric

Total: **100 points**

Design and API Contract, 15 pts

Criteria	Points
<code>openapi.yaml</code> hand written and committed in Sprint 1, before controllers	5
ER diagram with keys, cardinality, and nullability	3
Auth sequence diagram showing the full PKCE flow	2
Consistent error model, documented and used by every endpoint	2
Pagination on a collection, plus versioning, with reasoning documented	2
Two ADRs	1

REST API Implementation, 16 pts

Criteria	Points
All five verbs implemented with meaningful, non-trivial routes	7
Correct HTTP status codes throughout	3
Admin role with full user management endpoints	4
Implementation matches the committed contract	2

Testing, 20 pts

Criteria	Points
JUnit 5 service layer tests at 70 percent line coverage, enforced by the build	6
Tests are meaningful: business logic and edge cases, not getters	3
REST Assured integration tests covering happy, 401, 403, 404, and 400 paths	5
Integration tests run against Testcontainers	2
Android tests: ViewModel, MockWebServer, and one Compose UI test	3
GitHub Actions runs tests on every PR as a required check	1

Authentication, 10 pts

Criteria	Points
OAuth2 PKCE login and logout working from the Android app against the live API	5
API configured as a Resource Server and validating tokens	2
Protected routes return 401 and 403 correctly	2
Admin role stored in the database and enforced server side	1

Agile Process, 10 pts

Criteria	Points
GitHub Project board in use, issues linked to milestones	2

Criteria	Points
Issues written as user stories with clear acceptance criteria	3
All PRs reviewed and approved before merge	3
Four sprint retrospectives submitted	2

Docker and Deployment, 10 pts

Criteria	Points
Multi-stage Dockerfile and <code>.dockerignore</code> , image builds successfully	2
<code>docker-compose.yml</code> starts the API and a local database in one command	2
CI builds the image on every PR, and publishes to GHCR on merge to main	2
API live and reachable at a public HTTPS URL	4

Testcontainers is scored under Testing, and secrets handling under Documentation and this category jointly: a credential committed to the repo or baked into the image loses the Dockerfile points outright.

Database and Persistence, 7 pts

Criteria	Points
Remote hosted database, not local and not bundled in Docker	2
At least three tables or collections	2
At least one meaningful relationship	2
User data fully deleted on account deletion	1

Android Front End, 6 pts

Criteria	Points
All four screens working against the live API	3
OAuth2 login and logout work in the app	2
Error states surfaced for 401, 403, 404, and 500	1

Documentation, 6 pts

Criteria	Points
Swagger UI live, controllers annotated	2
README complete in each repo	2
KDoc or Javadoc on all public classes and methods	2

Helpful Links

Every link below was checked on 2026-09-22.

Spring Boot

Resource	Link
Spring Boot quickstart	https://spring.io/quickstart
Building a REST service	https://spring.io/guides/gs/rest-service/
Spring Data JPA	https://spring.io/guides/gs/accessing-data-jpa/
SQL database reference	https://docs.spring.io/spring-boot/reference/data/sql.html
Profiles	https://docs.spring.io/spring-boot/reference/features/profiles.html
Error handling, Problem Details	https://docs.spring.io/spring-framework/reference/web/webmvc/mvc-ann-rest-exceptions.html

OAuth2 and PKCE

Resource	Link
What PKCE is	https://oauth.net/2/pkce/
RFC 7636, the PKCE spec	https://datatracker.ietf.org/doc/html/rfc7636
Google OAuth2 for native apps	https://developers.google.com/identity/protocols/oauth2/native-app
Google OAuth2 setup	https://developers.google.com/identity/protocols/oauth2
Spring Security Resource Server, JWT	https://docs.spring.io/spring-security/reference/servlet/oauth2/resource-server/jwt.html
Resource Server overview	https://docs.spring.io/spring-security/reference/servlet/oauth2/resource-server/index.html
Spring Boot security	https://docs.spring.io/spring-boot/reference/web/spring-security.html
Method level security	https://docs.spring.io/spring-security/reference/servlet/authorization/method-security.html
AppAuth for Android	https://github.com/openid/AppAuth-Android
Credential Manager, Sign in with Google	https://developer.android.com/identity/sign-in/credential-manager-siwg

Android and Kotlin

Resource	Link
Compose documentation	https://developer.android.com/develop/ui/compose/documentation
Android Basics with Compose course	https://developer.android.com/courses/android-basics-compose/course
Recommended app architecture	https://developer.android.com/topic/architecture
Connecting to the network	https://developer.android.com/training/basics/network-ops/connecting
Emulator networking, the 10.0.2.2 rule	https://developer.android.com/studio/run/emulator-networking
Network security configuration	https://developer.android.com/privacy-and-security/security-config
Retrofit	https://github.com/square/retrofit
OkHttp	https://github.com/square/okhttp
Kotlin coroutines guide	https://kotlinlang.org/docs/coroutines-guide.html

Testing

Resource	Link
JUnit 5 user guide	https://junit.org/junit5/docs/current/user-guide/
Mockito	https://site.mockito.org/
Spring Boot testing reference	https://docs.spring.io/spring-boot/reference/testing/index.html
Testing Spring Boot applications	https://docs.spring.io/spring-boot/reference/testing/spring-boot-applications.html
Testing the web layer	https://spring.io/guides/gs/testing-web/
REST Assured	https://rest-assured.io/
REST Assured usage guide	https://github.com/rest-assured/rest-assured/wiki/Usage
Testcontainers for Java	https://java.testcontainers.org/
Testcontainers Postgres module	https://java.testcontainers.org/modules/databases/postgres/
Spring Boot Testcontainers support	https://docs.spring.io/spring-boot/reference/testing/testcontainers.html
JaCoCo Gradle plugin	https://docs.gradle.org/current/userguide/jacoco_plugin.html
MockWebServer	https://github.com/square/okhttp/tree/master/mockwebserver
Compose testing	https://developer.android.com/develop/ui/compose/testing
Compose testing cheat sheet	https://developer.android.com/develop/ui/compose/testing/testing-cheatsheet
Coroutines testing	https://developer.android.com/kotlin/coroutines/test
Android local unit tests	https://developer.android.com/training/testing/local-tests

Resource	Link
GitHub Actions	https://docs.github.com/en/actions
Build and test Java with Gradle	https://docs.github.com/en/actions/tutorials/build-and-test-code/java-with-gradle

Design and Documentation

Resource	Link
OpenAPI specification	https://swagger.io/specification/
Swagger Editor	https://editor.swagger.io/
SpringDoc OpenAPI	https://springdoc.org/
RFC 9457, Problem Details	https://www.rfc-editor.org/rfc/rfc9457
HTTP status codes	https://developer.mozilla.org/en-US/docs/Web/HTTP/Status
Mermaid ER diagrams	https://mermaid.js.org/syntax/entityRelationshipDiagram.html
Mermaid sequence diagrams	https://mermaid.js.org/syntax/sequenceDiagram.html
dbdiagram.io	https://dbdiagram.io/
Architecture Decision Records	https://adr.github.io/
ADR templates and examples	https://github.com/joelparkerhenderson/architecture-decision-record

Docker, Hosting, and Tooling

Resource	Link
Docker getting started	https://docs.docker.com/get-started/
Dockerfile reference	https://docs.docker.com/reference/dockerfile/
Multi-stage builds	https://docs.docker.com/build/building/multi-stage/
Dockerfile best practices	https://docs.docker.com/build/building/best-practices/
<code>.dockerignore</code> files	https://docs.docker.com/build/concepts/context/#dockerignore-files
Docker Compose getting started	https://docs.docker.com/compose/gettingstarted/
Spring Boot with Docker	https://spring.io/guides/gs/spring-boot-docker/
Publishing images with GitHub Actions	https://docs.github.com/en/packages/ managing-github-packages-using-github-actions-workflows/publishing-and-installing-a-package-with-github-actions
docker/build-push-action	https://github.com/docker/build-push-action
GitHub Actions secrets	https://docs.github.com/en/actions/concepts/security/secrets
Spring Actuator endpoints	https://docs.spring.io/spring-boot/reference/actuator/endpoints.html
Gradle guides	https://gradle.org/guides/
GitHub Projects	https://docs.github.com/en/issues/planning-and-tracking-with-projects/learning-about-projects/about-projects
Supabase	https://supabase.com/
MongoDB Atlas	https://www.mongodb.com/cloud/atlas
Railway	https://railway.app/

Resource	Link
Render	https://render.com/
Postman	https://www.postman.com/

Frequently Asked Questions

Does the domain have to be one of the five listed? No, but a proposal needs approval in Week 1.

Do we have to use a database? Yes, and it has to be remote and hosted.

Can we use Firebase? Firestore is fine as the database. Do not use Firebase for your API routes, and you do not need Firebase Auth since you are doing OAuth2 through Spring Security.

Do we have to use Google for OAuth2? No. GitHub and Auth0 are both fine. Ask first for anything else. Whatever you pick, it has to work with the Authorization Code plus PKCE flow from a native app.

Why can't we just do the OAuth flow the way the Spring tutorial shows? Most Spring OAuth2 tutorials configure the server as an OAuth2 Client with a session cookie, which assumes a browser. A native Android app has no session cookie to send. Your API is a Resource Server that validates a Bearer token. See the Authentication section.

Do we have to use Docker? Yes, in four places: a multi-stage build for the API, a compose file for local development, Testcontainers in your integration tests, and an image build in CI. See the Docker section.

Our integration tests fail in CI with a Docker error. Testcontainers needs a Docker daemon on the runner. The GitHub Actions `ubuntu-latest` runner has one. If you are on a self hosted runner or a Mac runner, you will need to provide it.

Does the Android app have to be on the Play Store? No. Turn in a debug APK on a GitHub release, or a screen recording against your live API.

Can we use a different backend framework? No. Spring Boot is required.

Can we build a web front end instead of Android? No. Kotlin Android is required this term.

Can we add more than four screens? You can, but they are worth zero points and I would rather you spend the time on tests. The API is the deliverable.

What counts as a meaningful unit test? One that verifies a decision your code makes. Does the service reject invalid input? Does it refuse to return a resource belonging to a different user? Does it handle an empty collection? A test that sets a field and asserts the getter returns it is not meaningful.

How is the 70 percent coverage floor measured? JaCoCo line coverage on your service layer package, checked by `jacocoTestCoverageVerification` in your Gradle build. If the build fails, CI fails, and the PR does not merge.

What if our login breaks during the demo? Test the live deployment before demo day. Have a recording as a backup. Seriously.

Changelog

Sep 22, 2026 v2.0.0,

- Front end changed from React to **Kotlin Android with Compose**, scoped down from seven screens to four
- Authentication rewritten for **Authorization Code with PKCE**; the API is now a **Resource Server**, not an OAuth2 Client
- **CORS requirement removed**. It is a browser policy and does not apply to a native client
- New **Design and API Contract** category, 15 pts: contract first OpenAPI, ER diagram, auth sequence diagram, error model, pagination and versioning, ADRs
- Design review added at the end of Sprint 1
- **Testing raised from 15 to 20 pts**, with a measurable coverage floor, Testcontainers, Android tests, and CI moved to run on pull requests
- **Docker deepened from a deploy artifact to a working tool**, 8 pts to 10: multi-stage build, `docker-compose.yml` for local development, image built in CI on every PR and published to GHCR on merge, secrets kept out of the image, and Testcontainers named explicitly as Docker. REST API Implementation moved 18 to 16 to fund it
- Postman collection requirement removed; superseded by the OpenAPI contract
- Gradle standardized across both repos
- Auth slice moved into Sprint 2 and integration tests into Sprint 3, so Sprint 4 is not a crunch
- Issue count restated as 5 per team member

v1.1.0, Feb 19 2026. Added grading rubric and helpful links. **v1.0.0, Feb 2026**. Created the assignment.